ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 28

Primeros pasos con Laravel

- Qué es Laravel
- Instalar Laravel
- La estructura del proyecto
- El ciclo de vida de una petición
- La configuración y el archivo .env
- Nuestras primeras rutas y vistas
- Artisan, la navaja suiza
- El frontend con Vite



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Llevamos cinco clases construyendo nuestro propio *framework*: front controller, router, peticiones y respuestas, modelos, vistas, contenedor, middleware, validación, autenticación y API. Os lo dije en la clase 26: todo eso lo tiene **Laravel**, con más opciones y años de pulido por detrás. Hoy por fin lo instalamos.

Y vais a tener una sensación muy curiosa: la de abrir un proyecto enorme y **reconocer las piezas**. Esa es la gran ventaja de haber hecho el camino largo. 😉

•
👌 Cómo vamos a trabajar



1
📘 Qué es Laravel



2
👌 Instalar Laravel



3
🔍 La estructura del proyecto



4
🤔 El ciclo de vida de una petición



5
📘 La configuración y el archivo .env



6
👍 Nuestras primeras rutas y vistas



7
😮 Artisan, la navaja suiza



8
⚠️ El frontend con Vite



9
🚀 Ejercicios



10
👋 Lo que tenéis que llevaros de hoy





CLASE 28

👌 Cómo vamos a trabajar

Durante todo el bloque de Laravel vamos a construir **una sola aplicación**: la web de la academia, que llamaremos `academia-laravel`. Hoy la creamos y, en cada clase, le añadimos piezas.

En la carpeta de cada clase encontraréis:

- `ejemplos/`: los ficheros que tocamos en la clase, **con la misma ruta que tienen dentro del proyecto** (`routes/web.php`, `resources/views/...`). Para ponerlos al día, copiad su contenido encima de vuestro proyecto.
- `soluciones/`: las soluciones de los ejercicios, organizadas igual.



⚠️ Nunca copiéis el `.env` ni la carpeta `vendor/` de un proyecto a otro: cada instalación tiene los suyos.



CLASE 28 · PUNTO 1

📘 Qué es Laravel

Laravel es un *framework* de PHP de código abierto creado por Taylor Otwell en 2011. Hoy es el más usado del mundo PHP, y no por casualidad [1]:

- Sigue la arquitectura **MVC** que ya conocéis.
- Trae resuelto lo que todas las webs necesitan: rutas, base de datos, validación, sesiones, autenticación, correo, colas, caché, pruebas…
- Tiene una documentación excelente y una comunidad enorme (si tenéis una duda, alguien la tuvo antes).
- Saca una versión principal **cada año**. Nosotros usamos **Laravel 13**, que necesita **PHP 8.3** o superior.

¿Por qué no empezamos el curso directamente con Laravel? Porque un *framework* esconde mucho trabajo. Si no sabéis qué hay debajo, cualquier error os parece magia negra. Vosotros sí lo sabéis: cuando Laravel haga algo «solo», pensad en qué clase de nuestro MVC lo hacía.



CLASE 28 · PUNTO 2

👌 Instalar Laravel

Lo que necesitamos

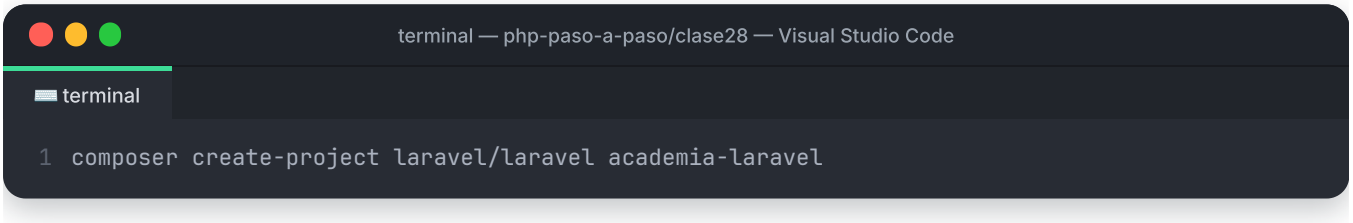
Herramienta	Para qué	¿La tenemos?
PHP 8.3 o superior	Laravel 13 lo exige	✅ Laragon (comprobad la versión con `php -v`)
Composer	Descarga Laravel y sus paquetes	✅ Viene con Laragon (clase 20)
MySQL	La base de datos	✅ Laragon + HeidiSQL
Node.js y npm	Solo para Vite (punto 8)	Opcional por ahora

Opción A: desde Laragon

1. Clic derecho en Laragon → **Inicio rápido** (*Quick app*) → **Laravel**.
2. Le damos el nombre `academia-laravel`.
3. Laragon ejecuta Composer, crea la carpeta en `C:\laragon\www\academia-laravel` y la base de datos, y además crea el **host virtual** `http://academia-laravel.test` apuntando a la carpeta `public/`. Recordad que, si no aparece, hay que **recargar** Apache.

Opción B: desde la terminal

Abrimos la terminal de Laragon (el botón **Terminal**) dentro de `C:\laragon\www` y escribimos:



Composer descarga Laravel con todas sus dependencias (unos minutos, paciencia ☕), crea el archivo `.env` y genera la clave de la aplicación. Al terminar, recargamos Laragon para que cree el host virtual.



💡 También existe el instalador oficial, `laravel new academia-laravel`, que pregunta si queremos un *starter kit* (los veremos en la clase 34). Para empezar, `composer create-project` nos da el proyecto limpio.

La base de datos

Por defecto, Laravel 13 usa **SQLite** (una base de datos en un fichero). Nosotros vamos a seguir con **MySQL**: creamos en HeidiSQL la base de datos `academia_laravel` (cotejamiento `utf8mb4_unicode_ci`) y cambiamos estas líneas del `.env` (en el punto 5 explico qué es):

ejemplo.php — php-paso-a-paso/clase28 — Visual Studio Code

ejemplo.php

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=academia_laravel
DB_USERNAME=root
DB_PASSWORD=
```

Después, desde la carpeta del proyecto:

terminal — php-paso-a-paso/clase28 — Visual Studio Code

terminal

```
php artisan migrate
```

Esto crea las tablas que Laravel necesita para funcionar (usuarios, sesiones, caché, colas). En la clase 31 veremos las migraciones a fondo.

Abrid `http://academia-laravel.test` en el navegador. Si veis la página de bienvenida de Laravel, **¡enhorabuena!** 🎉



🆘 **¿No sale?** Si `.test` no funciona, en la terminal ejecutad `php artisan serve` y abrid `http://127.0.0.1:8000`. Es un servidor de desarrollo que trae el propio Laravel.



CLASE 28 · PUNTO 3

🔍 La estructura del proyecto

La primera vez que se abre un proyecto de Laravel en VS Code impresiona: hay muchas carpetas. Tranquilos, en el día a día trabajaremos en cuatro o cinco.

Esquema

```
academia-laravel/
├── app/                    ← NUESTRO CÓDIGO PHP
│   ├── Http/Controllers/   ← los controladores (clase 29)
│   ├── Models/             ← los modelos de Eloquent (clase 32)
│   └── Providers/          ← los service providers (clase 35)
├── bootstrap/app.php       ← arranca la aplicación: rutas, middleware y errores
├── config/                 ← la configuración (un fichero por tema)
├── database/
│   ├── migrations/         ← la estructura de las tablas (clase 31)
│   ├── factories/          ← datos falsos para pruebas
│   └── seeders/            ← datos iniciales
├── public/                 ← LA ÚNICA CARPETA VISIBLE DESDE LA WEB
│   └── index.php           ← el front controller
├── resources/
│   ├── views/              ← las vistas Blade (clase 30)
│   ├── css/  js/           ← el CSS y el JS que procesa Vite
├── routes/
│   ├── web.php             ← las rutas de la web
│   └── console.php         ← los comandos de consola
├── storage/                ← logs, caché, sesiones y archivos subidos
├── tests/                  ← las pruebas (clase 37)
├── vendor/                 ← los paquetes de Composer (¡no se toca!)
├── .env                    ← la configuración de ESTE ordenador
├── artisan                 ← la herramienta de consola
└── composer.json           ← las dependencias
```

¿Os suena? Comparadlo con nuestro `academia-mvc`:

Nuestro MVC	Laravel
`public/index.php`	`public/index.php`
`config/rutas.php`	`routes/web.php`
`src/Controladores/`	`app/Http/Controllers/`
`src/Modelos/`	`app/Models/`
`vistas/`	`resources/views/`
`config/servicios.php`	`app/Providers/`
`.env`	`.env`
`vendor/`	`vendor/`



⚠️ El servidor web tiene que apuntar a la carpeta `public/`, igual que en nuestro MVC. Si apuntara a la raíz, cualquiera podría descargar vuestro `.env` con las contraseñas.



CLASE 28 · PUNTO 4

🤔 El ciclo de vida de una petición

¿Qué pasa desde que escribimos `http://academia-laravel.test/hola` hasta que vemos la página? Lo mismo que en nuestro MVC, con más pasos intermedios [2]:

Esquema

```
Navegador: GET /hola
      │
      ▼
public/.htaccess ──► public/index.php          (front controller, clase 23)
      │
      ▼
bootstrap/app.php: crea la aplicación          (nuestra clase Aplicacion)
      │   carga .env y config/, registra los service providers
      ▼
Kernel HTTP: convierte la petición en un objeto Request   (Peticion, clase 24)
      │
      ▼
Middleware: sesión, cookies, CSRF…             (Middleware, clase 26)
      │
      ▼
Router: busca la ruta GET /hola en routes/web.php   (Router, clase 24)
      │
      ▼
Controlador o función ──► Vista Blade / modelo  (clase 25)
      │
      ▼
Response ──► vuelve por el middleware ──► Navegador   (Respuesta, clase 24)
```

Abrid `public/index.php` de Laravel: son unas pocas líneas que cargan el autoload de Composer, crean la aplicación con `bootstrap/app.php` y le pasan la petición. Exactamente lo que hacía el nuestro.



CLASE 28 · PUNTO 5

📘 La configuración y el archivo .env

El archivo .env

Igual que en la clase 25, la configuración que **cambia de un ordenador a otro** (contraseñas, direcciones, modo de depuración) va en el `.env`. Las líneas más importantes:

ejemplo.php — php-paso-a-paso/clase28 — Visual Studio Code

ejemplo.php

```
APP_NAME="Academia Alpe"
APP_ENV=local
APP_KEY=base64:...
APP_DEBUG=true
APP_URL=http://academia-laravel.test

APP_LOCALE=es
APP_FALLBACK_LOCALE=en
APP_FAKER_LOCALE=es_ES
```

Variable	Qué hace
`APP_NAME`	El nombre de la aplicación (sale en títulos y correos). Con espacios, entre comillas
`APP_ENV`	El entorno: `local` en nuestro ordenador, `production` en el servidor
`APP_KEY`	La clave para cifrar sesiones y cookies. La genera `php artisan key:generate`. **Nunca** la compartáis
`APP_DEBUG`	`true` muestra los errores con detalle; en producción, **siempre** `false`
`APP_LOCALE`	El idioma de la aplicación: `es`
`APP_FAKER_LOCALE`	El idioma de los datos falsos (clase 31): nombres españoles



⚠️ El `.env` **no se sube a Git** (ya está en el `.gitignore`). Lo que sí se sube es el `.env.example`, una plantilla sin contraseñas para que el compañero sepa qué variables necesita.

La carpeta config/

Cada fichero de `config/` devuelve un array con la configuración de un tema: `app.php`, `database.php`, `mail.php`... Muchos valores se leen del `.env` con `env()`:

ejemplo.php — php-paso-a-paso/clase28 — Visual Studio Code

ejemplo.php

```
// config/app.php
'name' => env('APP_NAME', 'Laravel'),
'locale' => env('APP_LOCALE', 'en'),
'timezone' => 'UTC',
```

La zona horaria no está en el `.env`, así que la cambiamos en el propio `config/app.php`, porque estamos en España:

ejemplo.php — php-paso-a-paso/clase28 — Visual Studio Code

ejemplo.php

```
'timezone' => 'Europe/Madrid',
```

En nuestro código **nunca** usamos `env()`: leemos la configuración con `config()` y la notación de puntos (fichero.clave):

ejemplo.php — php-paso-a-paso/clase28 — Visual Studio Code

ejemplo.php

```
config('app.name');       // "Academia Alpe"
config('app.timezone');   // "Europe/Madrid"
config('database.connections.mysql.database');   // "academia_laravel"
```

¿Por qué? Porque en producción Laravel junta toda la configuración en un solo fichero de caché (`php artisan config:cache`) y, a partir de ahí, deja de leer el `.env`. Si usáis `env()` fuera de `config/`, os devolverá `null`. 😱



💡 Si cambiáis el `.env` y no notáis el cambio, ejecutad `php artisan config:clear`.



CLASE 28 · PUNTO 6

👍 Nuestras primeras rutas y vistas

Abrimos `routes/web.php`. Laravel trae una sola ruta: la de la página de bienvenida. Añadimos tres:

ejemplo.php — php-paso-a-paso/clase28 — Visual Studio Code

ejemplo.php

```php
<?php

use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

// Una ruta que devuelve un texto: Laravel lo convierte en la respuesta
Route::get('/hola', function () {
    return '¡Hola desde Laravel! 👋';
});

// Una ruta que devuelve una vista y le pasa datos
Route::get('/saludo', function () {
    return view('saludo', [
        'nombre' => 'Ramón',
        'curso' => 'PHP y POO',
    ]);
});
```

Fijaos en lo que **no** hemos tenido que hacer: ni `echo`, ni cabeceras, ni `require` de la vista. Si la función devuelve un texto, Laravel lo envía; si devuelve una vista, la pinta.

`view('saludo', [...])` busca el fichero `resources/views/saludo.blade.php` y le pasa los datos: cada clave del array se convierte en una variable dentro de la vista (`saludo.blade.php`):

plantilla.php — php-paso-a-paso/clase28 — Visual Studio Code

plantilla.php

```blade
<link rel="stylesheet" href="{{ asset('css/academia.css') }}">
...
<h1>¡Hola, {{ $nombre }}! 👋</h1>
<p>Bienvenido al curso de <strong>{{ $curso }}</strong>.</p>
```

- `{{ $nombre }}` es el `<?= e($nombre) ?>` de nuestro MVC: imprime **escapando** el HTML. Adiós a los ataques XSS sin pensar en ello. Es **Blade**, el motor de plantillas de Laravel, y le dedicamos la clase 30.
- `asset('css/academia.css')` genera la dirección completa de un fichero de `public/`: `http://academia-laravel.test/css/academia.css`.

La tercera ruta, `/info`, enseña los datos de la configuración en una tabla (`info.blade.php`):

ejemplo.php — php-paso-a-paso/clase28 — Visual Studio Code

ejemplo.php

```php
Route::get('/info', function () {
    return view('info', [
        'nombreApp' => config('app.name'),
        'entorno' => app()->environment(),
        'depuracion' => config('app.debug'),
        'idioma' => config('app.locale'),
        'zonaHoraria' => config('app.timezone'),
        'versionLaravel' => app()->version(),
        'versionPhp' => PHP_VERSION,
        'ahora' => now()->format('d/m/Y H:i'),
    ]);
});
```

`now()` devuelve la fecha y hora actuales como un objeto **Carbon**, una versión con superpoderes del `DateTime` de PHP que Laravel usa en todas partes.



Artisan es la herramienta de consola de Laravel: el fichero `artisan` de la raíz. Se usa siempre desde la carpeta del proyecto con `php artisan ...` [3]. Estos son los comandos que más vais a usar:

Comando	Qué hace
`php artisan list`	Lista todos los comandos (son muchos)
`php artisan help migrate`	Explica un comando y sus opciones
`php artisan about`	Resumen de la aplicación: versiones, entorno, caché…
`php artisan serve`	Servidor de desarrollo en `http://127.0.0.1:8000`
`php artisan route:list`	Todas las rutas de la aplicación
`php artisan make:controller CursoController`	Crea un controlador (hay `make:` para casi todo)
`php artisan migrate`	Ejecuta las migraciones pendientes
`php artisan tinker`	Consola interactiva para probar código PHP de la aplicación
`php artisan config:clear`	Borra la caché de configuración
`php artisan optimize:clear`	Borra **todas** las cachés (el «apagar y encender» de Laravel)

Tinker

`tinker` abre una consola donde podemos escribir PHP con toda la aplicación cargada. Es ideal para probar cosas rápidas:

Esquema

```
> config('app.name')
= "Academia Alpe"

> now()->addDays(10)->format('d/m/Y')
= "17/11/2026"

> exit
```

Nuestro primer comando

Igual que nosotros hicimos scripts en `herramientas/`, en Laravel podemos crear comandos propios. Los sencillos se escriben en `routes/console.php`:

ejemplo.php — php-paso-a-paso/clase28 — Visual Studio Code

ejemplo.php

```php
// Nuestro primer comando: php artisan academia:saludo Lucía
Artisan::command('academia:saludo {nombre=alumno}', function (string $nombre) {
    $this->info("¡Hola, {$nombre}! Bienvenido a " . config('app.name') . '.');
    $this->line('Hoy es ' . now()->format('d/m/Y') . '.');
})->purpose('Saluda a un alumno por la consola');
```

- `{nombre=alumno}` es un **argumento** con valor por defecto.
- `$this->info()` escribe en verde; `$this->line()`, en normal; `$this->error()`, en rojo.
- `purpose()` es la descripción que sale en `php artisan list`.

Esquema

```
$ php artisan academia:saludo Lucía
¡Hola, Lucía! Bienvenido a Academia Alpe.
Hoy es 07/11/2026.
```



💡 Usad siempre un prefijo propio (`academia:`) para vuestros comandos: así se agrupan en el `list` y no chocan con los de Laravel.



CLASE 28 · PUNTO 8

⚠️ El frontend con Vite

En una web moderna, el CSS y el JavaScript también se «compilan»: se juntan en pocos ficheros, se reducen de tamaño y se recargan solos al guardar. Laravel usa para eso **Vite**, una herramienta de JavaScript que funciona con **Node.js** [4].

Si tenéis Node instalado (Laragon lo incluye), el flujo es este:

terminal — php-paso-a-paso/clase28 — Visual Studio Code

terminal

```
npm install        # descarga las dependencias de JavaScript (una vez)
npm run dev        # modo desarrollo: recarga el navegador al guardar
npm run build      # versión final, optimizada, en public/build/
```

Y en la vista se cargan con la directiva `@vite`:

ejemplo.php — php-paso-a-paso/clase28 — Visual Studio Code

ejemplo.php

```
@vite(['resources/css/app.css', 'resources/js/app.js'])
```

Laravel 13 trae configurado **Tailwind CSS**, un *framework* de CSS muy popular, en `resources/css/app.css`.

Nuestra decisión para el curso

Para no depender de Node mientras aprendemos Laravel, vamos a usar una **hoja de estilos normal**: `public/css/academia.css`. Al estar en `public/`, se sirve tal cual y la enlazamos con `asset()`:

plantilla.php — php-paso-a-paso/clase28 — Visual Studio Code

plantilla.php

```
<link rel="stylesheet" href="{{ asset('css/academia.css') }}">
```

	`public/css` + `asset()`	Vite + `@vite`
Necesita Node	No	Sí
Recarga automática	No	Sí
Optimiza y junta ficheros	No	Sí
Ideal para…	Aprender y proyectos pequeños	Proyectos reales con mucho CSS y JS

Cuando os sintáis cómodos con Laravel, pasar a Vite es cambiar una línea del layout. Lo veremos otra vez en la clase 30.



CLASE 28 · PUNTO 9

🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Sobre mí.** Cread la ruta `/sobre-mi`, que muestre una vista con vuestro nombre, vuestra ciudad y vuestras aficiones, pasadas como datos desde la ruta. Usad la hoja de estilos de la academia. → `routes/web.php` y `sobre-mi.blade.php`
- ☐ **02 · Comando de información.** Cread el comando `php artisan academia:info`, que muestre en una tabla el entorno, la versión de Laravel, la de PHP, el nombre de la base de datos y el idioma. Pista: `$this->table(['Dato', 'Valor'], [...])`. → `routes/console.php`
- ☐ **03 · ¿Qué día es hoy?** Cread la ruta `/hoy`, que devuelva un texto como «Hoy es sábado, 7 de noviembre de 2026 y son las 21:26 en Europe/Madrid.». Pista: `now()->locale('es')->isoFormat('dddd, D [de] MMMM [de] YYYY')`. → `routes/web.php`
- ☐ **04 · La versión en JSON.** La ruta `/version` devuelve un **array** con el nombre de la aplicación, la versión de Laravel, la de PHP y el entorno. Abridla: ¿en qué formato sale? → `routes/web.php`
- ☐ **05 · Contacto rápido.** Una vista `contacto-rapido` con el nombre de la aplicación, su dirección (`config('app.url')`), un teléfono y un correo, todo pasado desde la ruta. → `routes/web.php` y `contacto-rapido.blade.php`
- ☐ **06 · Un fichero de configuración propio.** Cread `config/academia.php` con el teléfono y el correo (leyéndolos del `.env` con un valor por defecto) y usadlos en la vista anterior con `config('academia.telefono')`. → `config/academia.php`
- ☐ **07 · Sumar por la consola.** `php artisan academia:sumar 4 7`, con dos argumentos y un error en rojo si alguno no es un número. → `routes/console.php`
- ☐ **08 · Tablas de multiplicar.** `php artisan academia:tabla 7` muestra la tabla con `$this->table()`; sin número, la del 1. → `routes/console.php`
- ☐ **09 · Un comando que pregunta.** `php artisan academia:ficha` pide el nombre (`$this->ask()`), un curso de una lista (`$this->choice()`) y si quiere avisos (`$this->confirm()`), y muestra la ficha. → `routes/console.php`
- ☐ **10 · Limpieza general.** `php artisan academia:limpiar` llama a `optimize:clear` con `$this->call()` y avisa de que las cachés están limpias. → `routes/console.php`



🏆 **Reto:** ejecutad `php artisan route:list` y `php artisan about`, y explicad con vuestras palabras cada línea que salga. Después cambiad `APP_DEBUG` a `false`, provocad un error (una ruta que llame a una función que no existe) y comparad la página de error con la que sale con `APP_DEBUG=true`.



CLASE 28 · PUNTO 10

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Laravel	El *framework* PHP más usado: un MVC como el nuestro, con todo resuelto
Instalación	`composer create-project laravel/laravel academia-laravel` (o Inicio rápido de Laragon)
`public/`	La única carpeta visible desde la web; ahí está el front controller
`.env`	La configuración de cada ordenador; nunca se sube a Git
`config()`	Cómo se lee la configuración en el código (`env()` solo dentro de `config/`)
`routes/web.php`	Donde se declaran las rutas de la web
`view()`	Devuelve una vista Blade con los datos que le pasemos
`{{ }}`	Imprime escapando el HTML
Artisan	La consola de Laravel: `make:` , `migrate` , `route:list` , `tinker` …
Vite	Compila el CSS y el JS; de momento usamos `public/css` con `asset()`

📚 Fuentes y documentación oficial

1. Laravel: Installation
2. Laravel: Request Lifecycle
3. Laravel: Artisan Console
4. Laravel: Asset Bundling (Vite)
5. Laravel: Configuration
6. Carbon: documentación